

Kotlin & Jetpack Compose Developer Guide

Volume 5

Per-Item State in Lists | Dynamic DI | @AssistedInject
ViewModel per Card (Anti-Pattern) | Flutter vs Kotlin Comparison

For Flutter/Dart developers learning Kotlin & Jetpack Compose

Chapter 1: Per-Item State in Lists

The Question

Imagine you have a `ProductCard` composable and you want each card to show its own `CircularProgressIndicator` while being deleted. The question: should each `ProductCard` create its own `ViewModel` by calling `hiltViewModel()` inside the card composable?

Why a ViewModel Per Card Does NOT Work

[!] `hiltViewModel()` scopes the `ViewModel` to the nearest navigation destination (screen). ALL cards calling `hiltViewModel<ProductDeletionViewModel>()` get the EXACT SAME instance -- not a separate one per card. This means one loading state affects ALL cards simultaneously.

```
// WRONG approach
@Composable
fun ProductCard(
    product: Product,
    onClick: () -> Unit,
    viewModel: ProductDeletionViewModel = hiltViewModel() // WRONG
) {
    // All ProductCards share the same ViewModel instance!
    // Deleting product 1 shows spinner on ALL cards, not just card 1.
}

// What actually happens:
ProductCard(product = p1) -> hiltViewModel() -> SAME ProductDeletionViewModel instance
ProductCard(product = p2) -> hiltViewModel() -> SAME ProductDeletionViewModel instance
ProductCard(product = p3) -> hiltViewModel() -> SAME ProductDeletionViewModel instance
```

The Correct Approach -- One ViewModel, a Set of Loading IDs

The screen-level `ViewModel` tracks which product IDs are currently being deleted using a `Set<Int>`. Each card receives its own `isDeleting`: Boolean derived from that set.

Kotlin -- ViewModel with deletingIds Set:

Kotlin / Jetpack Compose

```
// ProductsViewModel.kt
data class ProductsUiState(
    val products: List<Product>,
    val deletingIds: Set<Int> = emptySet(), // tracks which cards are loading
)

fun deleteProduct(productId: Int) {
    viewModelScope.launch {
```

Kotlin & Jetpack Compose Guide - Vol 5

```
// Add to loading set -- card shows spinner
_state.update { it.copy(deletingIds = it.deletingIds + productId) }
try {
    productRepository.deleteProduct(productId)
    // Remove from list AND loading set
    _state.update { it.copy(
        products = it.products.filter { p -> p.id != productId },
        deletingIds = it.deletingIds - productId,
    )}
} catch (e: Exception) {
    // Remove from loading set on error
    _state.update { it.copy(deletingIds = it.deletingIds - productId) }
}
}
```

Kotlin -- Screen passes per-card state down:

Kotlin / Jetpack Compose

```
// ProductsScreen.kt
items(currentState.products) { product ->
    ProductCard(
        product = product,
        isDeleting = product.id in currentState.deletingIds, // per-card boolean
        onDelete = { viewModel.deleteProduct(product.id) },
        onClick = { onProductClick(product.id) },
    )
}

// ProductCard.kt -- dumb component, no ViewModel
@Composable
fun ProductCard(
    product: Product,
    isDeleting: Boolean, // receives state, does not own it
    onDelete: () -> Unit,
    onClick: () -> Unit,
) {
    if (isDeleting) {
        CircularProgressIndicator() // only THIS card shows spinner
    } else {
        // normal card UI with delete button
        Button(onClick = onDelete) { Text("Delete") }
    }
}
```

Flutter equivalent -- same pattern with a Set in the controller:

Dart / Flutter

```
// products_controller.dart
```

```
class ProductsController extends ChangeNotifier {
    List<Product> products = [];
    Set<int> deletingIds = {}; // tracks which cards are loading

    Future<void> deleteProduct(int productId) async {
        deletingIds.add(productId); // card shows spinner
        notifyListeners();
        try {
            await productRepository.deleteProduct(productId);
            products.removeWhere((p) => p.id == productId);
            deletingIds.remove(productId);
        } catch (e) {
            deletingIds.remove(productId);
        }
        notifyListeners();
    }
}

// product_card.dart -- dumb widget
class ProductCard extends StatelessWidget {
    final Product product;
    final bool isDeleting; // receives state, does not own it
    final VoidCallback onDelete;

    @override
    Widget build(BuildContext context) {
        if (isDeleting) return CircularProgressIndicator();
        return ElevatedButton(onPressed: onDelete, child: Text('Delete'));
    }
}
```

The Rule

[OK] Composable components (ProductCard, CartItemCard, etc.) should be DUMB -- they receive data and callbacks, they NEVER own a ViewModel. ViewModels belong at the SCREEN level only.

Approach	Works?	Why
hiltViewModel() per card composable	No	All cards share ONE instance -- shared loading state
Screen ViewModel + Set<Int> of loading IDs	Yes	Each card gets its own isDeleting boolean

Chapter 2: Dynamic Dependency Injection -- Three Approaches

The Problem

You have one ViewModel and one interface (IOrderRepository) with two implementations. You want Screen A to use OrderRepositoryImpl (remote/network) and Screen B to use OrderLocalRepositoryImpl (local/instant). How do you load the correct implementation dynamically?

Option 1 -- Inject Both, Select via SavedStateHandle (Simplest)

Inject both implementations into the ViewModel constructor using @Named. Pass which one to use as a navigation argument. ViewModel reads it from SavedStateHandle.

Kotlin / Jetpack Compose

```
// AppModule.kt -- bind both with @Named
@Binds @Singleton @Named("Remote")
abstract fun bindRemote(impl: OrderRepositoryImpl): IOrderRepository

@Binds @Singleton @Named("Local")
abstract fun bindLocal(impl: OrderLocalRepositoryImpl): IOrderRepository

// OrderViewModel.kt
@HiltViewModel
class OrderViewModel @Inject constructor(
    @Named("Remote") private val remoteRepo: IOrderRepository,
    @Named("Local") private val localRepo: IOrderRepository,
    private val savedStateHandle: SavedStateHandle,
) : ViewModel() {
    // reads nav argument to pick which repo to use
    private val repository = if (savedStateHandle.get<Boolean>("useRemote") == true)
        remoteRepo else localRepo
}

// Navigation -- pass argument when navigating
navController.navigate("order?useRemote=true") // Screen A -- remote
navController.navigate("order?useRemote=false") // Screen B -- local
```

Dart / Flutter

```
// Flutter equivalent -- pass repo via constructor or Provider
// Screen A
Provider<IOrderRepository>(
    create: (_) => OrderRemoteRepositoryImpl(),
    child: OrderScreen(),
)

// Screen B
```

Kotlin & Jetpack Compose Guide - Vol 5

```
Provider<IOrderRepository>(  
    create: { _ } => OrderLocalRepositoryImpl(),  
    child: OrderScreen(),  
)
```

Option 2 -- @AssistedInject (The Proper DI Way -- What You Implemented)

Hilt injects everything it knows about (ICartRepository etc.), but the repository is passed BY THE CALLER at runtime. The screen decides which implementation to pass when creating the ViewModel via a factory.

Kotlin / Jetpack Compose

```
// OrderViewModel.kt  
@AssistedFactory  
interface OrderViewModelFactory {  
    fun create(orderRepository: IOrderRepository): OrderViewModel  
}  
  
@HiltViewModel(assistedFactory = OrderViewModelFactory::class)  
class OrderViewModel @AssistedInject constructor(  
    @Assisted private val orderRepository: IOrderRepository, // caller provides this  
    private val cartRepository: ICartRepository,             // Hilt provides this  
) : ViewModel()  
  
// AppModule.kt -- NO bindings needed for IOrderRepository!  
// (nothing to add -- @AssistedInject bypasses AppModule for the assisted param)  
  
// Screen A -- passes remote implementation  
val viewModel = hiltViewModel<OrderViewModel, OrderViewModelFactory> { factory ->  
    factory.create(OrderRepositoryImpl()) // remote -- 4 second delay  
}  
  
// Screen B -- passes local implementation  
val viewModel = hiltViewModel<OrderViewModel, OrderViewModelFactory> { factory ->  
    factory.create(OrderLocalRepositoryImpl()) // local -- instant  
}
```

Dart / Flutter

```
// Flutter closest equivalent -- factory constructor or abstract factory pattern  
// There is no direct AssistedInject in Flutter/Dart, but the same result  
// can be achieved by passing the repository directly to the controller:  
  
class OrderController extends ChangeNotifier {  
    final IOrderRepository repository; // passed by caller  
    OrderController({required this.repository});  
}  
  
// Screen A  
ChangeNotifierProvider(  
    create: (context) => OrderController(repository: OrderRepositoryImpl()),  
    child: OrderScreen(),  
)
```

Kotlin & Jetpack Compose Guide - Vol 5

```
        create: (_) => OrderController(repository: OrderRemoteRepositoryImpl()),
        child: OrderScreen(),
    )

// Screen B
ChangeNotifierProvider(
    create: (_) => OrderController(repository: OrderLocalRepositoryImpl()),
    child: OrderScreen(),
)
```

Option 3 -- Two Separate ViewModels (Most Explicit)

Create a base abstract ViewModel with all the logic. Two subclasses, each with their own @Named injection. Simple, no factory needed, very readable.

Kotlin / Jetpack Compose

```
// Base ViewModel -- contains all logic
abstract class OrderViewModel(
    private val repository: IOrderRepository
) : ViewModel() {
    fun placeOrder() { viewModelScope.launch { repository.placeOrder(...) } }
}

// Subclass 1 -- remote
@HiltViewModel
class RemoteOrderViewModel @Inject constructor(
    @Named("Remote") repository: IOrderRepository
) : OrderViewModel(repository)

// Subclass 2 -- local
@HiltViewModel
class LocalOrderViewModel @Inject constructor(
    @Named("Local") repository: IOrderRepository
) : OrderViewModel(repository)

// Screen A uses RemoteOrderViewModel
val viewModel: RemoteOrderViewModel = hiltViewModel()

// Screen B uses LocalOrderViewModel
val viewModel: LocalOrderViewModel = hiltViewModel()
```

Dart / Flutter

```
// Flutter equivalent -- two subclasses of the same controller
abstract class OrderController extends ChangeNotifier {
    final IOrderRepository repository;
    OrderController(this.repository);
    Future<void> placeOrder() async { await repository.placeOrder(...); }
}
```

```
class RemoteOrderController extends OrderController {  
    RemoteOrderController() : super(OrderRemoteRepositoryImpl());  
}  
  
class LocalOrderController extends OrderController {  
    LocalOrderController() : super(OrderLocalRepositoryImpl());  
}
```

Comparison of All Three Options

Option	Complexity	AppModule change?	Best for
SavedStateHandle + both	Low	Yes (@Named)	Simple bool flag from nav arg
@AssistedInject (your choice)	Medium	No	Passing any object at runtime
Two separate ViewModels	Low	Yes (@Named)	When impls behave differently

Chapter 3: @AssistedInject Deep Dive

The Core Concept

With standard Hilt injection, ALL constructor parameters are provided by Hilt (resolved at compile time via AppModule bindings). With @AssistedInject, you split the constructor into two groups:

```
class OrderViewModel @AssistedInject constructor(
    @Assisted private val orderRepository: IOrderRepository, // GROUP 1: YOU provide at runtime
    private val cartRepository: ICartRepository,             // GROUP 2: HILT provides from AppModule
)
```

[NOTE] Think of @AssistedInject as a partial constructor -- Hilt fills in what it knows, you fill in the rest when you call factory.create(...).

The Four Annotations Explained

Annotation	Where	What it does
@AssistedInject	constructor	Replaces @Inject -- tells Hilt this uses assisted
@Assisted	parameter	Marks which params the caller provides (not Hilt)
@AssistedFactory	interface	Hilt generates the implementation of this factory
@HiltViewModel(assistedFactory=...)	class	Links the ViewModel to its factory so hiltViewModel() can use it

Step-by-step: What Hilt Generates

When you write @AssistedFactory, Hilt generates the factory implementation at compile time:

```
// You write:
@AssistedFactory
interface OrderViewModelFactory {
    fun create(orderRepository: IOrderRepository): OrderViewModel
}

// Hilt generates (roughly):
class OrderViewModelFactory_Impl @Inject constructor(
    private val cartRepositoryProvider: Provider<ICartRepository>
) : OrderViewModelFactory {
    override fun create(orderRepository: IOrderRepository): OrderViewModel {
        return OrderViewModel(
            orderRepository = orderRepository,          // your runtime value
            cartRepository = cartRepositoryProvider.get() // Hilt's value from AppModule
        )
    }
}
```

How hiltViewModel() Uses the Factory

The overloaded hiltViewModel() with a lambda is the screen-side usage:

Kotlin / Jetpack Compose

```
// What you write in the screen:
val viewModel = hiltViewModel<OrderViewModel, OrderViewModelFactory> { factory ->
    factory.create(OrderRepositoryImpl())
}

// What happens step by step:
// 1. Hilt creates OrderViewModelFactory_Impl (injects cartRepository automatically)
// 2. hiltViewModel() calls your lambda with that factory instance
// 3. Your lambda calls factory.create(OrderRepositoryImpl()) -- passes the runtime repo
// 4. Factory combines: your OrderRepositoryImpl + Hilt's cartRepository
// 5. Returns a fully constructed OrderViewModel with both dependencies
```

Before vs After: Full Code Comparison

BEFORE -- standard @HiltViewModel (fixed implementation):

Kotlin / Jetpack Compose

```
// AppModule.kt -- must bind one implementation
@Binds @Singleton
abstract fun bindOrderRepository(impl: OrderRepositoryImpl): IOrderRepository

// OrderViewModel.kt
@HiltViewModel
class OrderViewModel @Inject constructor(
    private val orderRepository: IOrderRepository, // always OrderRepositoryImpl
    private val cartRepository: ICartRepository,
) : ViewModel()

// Screen -- hiltViewModel() with no control over which repo is used
val viewModel: OrderViewModel = hiltViewModel()
```

AFTER -- @AssistedInject (runtime implementation choice):

Kotlin / Jetpack Compose

```
// AppModule.kt -- NO binding needed for IOrderRepository
// (nothing to add)

// OrderViewModelFactory + OrderViewModel
@AssistedFactory
interface OrderViewModelFactory {
    fun create(orderRepository: IOrderRepository): OrderViewModel
}

@HiltViewModel(assistedFactory = OrderViewModelFactory::class)
```

```
class OrderViewModel @AssistedInject constructor(
    @Assisted private val orderRepository: IOrderRepository, // runtime choice
    private val cartRepository: ICartRepository,
) : ViewModel()

// Screen A -- remote (4 second delay)
val viewModel = hiltViewModel<OrderViewModel, OrderViewModelFactory> { factory ->
    factory.create(OrderRepositoryImpl())
}

// Screen B -- local (instant)
val viewModel = hiltViewModel<OrderViewModel, OrderViewModelFactory> { factory ->
    factory.create(OrderLocalRepositoryImpl())
}
```

Dart / Flutter

```
// Flutter equivalent of the AFTER pattern:
// Pass the repository to the controller constructor directly,
// and provide it via ChangeNotifierProvider in the parent widget.

class OrderController extends ChangeNotifier {
    final IOrderRepository repository; // injected by caller
    OrderController({required this.repository});

    Future<void> placeOrder(List<CartItem> items) async {
        await repository.placeOrder(items);
    }
}

// Screen A widget tree -- remote
ChangeNotifierProvider(
    create: (_) => OrderController(repository: OrderRemoteRepositoryImpl()),
    child: OrderScreen(),
)

// Screen B widget tree -- local
ChangeNotifierProvider(
    create: (_) => OrderController(repository: OrderLocalRepositoryImpl()),
    child: OrderScreen(),
)
```

Chapter 4: The Two Order Repository Implementations

OrderRepositoryImpl -- Remote / Network Simulation

Kotlin / Jetpack Compose

```
class OrderRepositoryImpl @Inject constructor() : IOrderRepository {

    private var orderIdCounter = 1

    override suspend fun placeOrder(items: List<CartItem>): Order {
        delay(4000) // simulates real network latency
        val total = items.sumOf { it.totalPrice }
        return Order(
            id = orderIdCounter++, // auto-incrementing ID like a real server
            items = items,
            totalPrice = total,
            status = OrderStatus.CONFIRMED,
        )
    }
}
```

Key characteristics:

```
delay(4000)           -- simulates 4-second network round-trip
orderIdCounter++      -- server-generated incrementing ID
@Inject constructor    -- Hilt can construct it when needed by the factory
```

OrderLocalRepositoryImpl -- Local / Offline Simulation

Kotlin / Jetpack Compose

```
class OrderLocalRepositoryImpl @Inject constructor() : IOrderRepository {
    override suspend fun placeOrder(items: List<CartItem>): Order {
        val total = items.sumOf { it.totalPrice }
        return Order(
            id = 100, // hardcoded local ID
            items = items,
            totalPrice = total,
            status = OrderStatus.CONFIRMED,
        )
    }
}
```

Key characteristics:

```
no delay              -- returns instantly (local database / in-memory)
id = 100               -- hardcoded local ID (not server-generated)
no network call        -- works offline
```

Dart / Flutter

```
// Dart equivalents

// Remote implementation
class OrderRemoteRepositoryImpl implements IOrderRepository {
    int _orderIdCounter = 1;

    @override
    Future<Order> placeOrder(List<CartItem> items) async {
        await Future.delayed(Duration(seconds: 4)); // simulates network
        final total = items.fold(0.0, (sum, i) => sum + i.totalPrice);
        return Order(id: _orderIdCounter++, items: items, totalPrice: total);
    }
}

// Local implementation
class OrderLocalRepositoryImpl implements IOrderRepository {
    @override
    Future<Order> placeOrder(List<CartItem> items) async {
        // no await -- returns instantly
        final total = items.fold(0.0, (sum, i) => sum + i.totalPrice);
        return Order(id: 100, items: items, totalPrice: total);
    }
}
```

Why Both Have @Inject constructor

Even though neither implementation is bound in AppModule via @Binds, both still have @Inject constructor. This is required so that Hilt can construct them when the @AssistedFactory calls factory.create(OrderRepositoryImpl()). Hilt needs to know how to build the object even if it is not bound to an interface.

[NOTE] @Inject constructor does not mean it MUST be registered in AppModule. It just means Hilt is ALLOWED to construct it. For @AssistedInject, you pass the already-constructed instance yourself -- factory.create(OrderRepositoryImpl()) -- so Hilt constructs it on demand without any AppModule binding.

Chapter 5: Quick Reference

Per-Item State Pattern

What you need	How to do it
Each list item has its own loading state	Set<Int> of loading IDs in screen ViewModel
Pass loading state to each card	isLoading = item.id in state.loadingIds
Card shows its own spinner	if (isLoading) CircularProgressIndicator()
Card has no ViewModel	Card receives callbacks only, never calls hiltViewModel()

When to Use Each Dynamic DI Option

Scenario	Best Option
Switch between 2 impls based on a boolean nav arg	Option 1: SavedStateHandle
Pass any runtime object to ViewModel	Option 2: @AssistedInject
Two screens, fundamentally different ViewModel behavior	Option 3: Two ViewModels
Single implementation, no switching needed	Standard @HiltViewModel + @Inject

@AssistedInject Cheat Sheet

Annotation	Purpose
@AssistedInject	Marks the constructor -- replaces @Inject
@Assisted	This param is provided by the caller, not Hilt
@AssistedFactory	Interface whose implementation Hilt generates
@HiltViewModel(assistedFactory=...)	Links ViewModel to its factory
hiltViewModel<VM, Factory> { f -> f.create(...) }	Screen-side usage -- pass runtime value

Flutter vs Kotlin Summary for This Chapter

Concept	Flutter / Dart	Kotlin / Jetpack
Per-item loading	Set<int> in ChangeNotifier, pass isLoading to	Set<Int> in ViewModel, pass isDeleting to
Card owns controller?	No -- StatelessWidget receives callbacks	No -- composable receives callbacks only
Pass repo at runtime	ChangeNotifierProvider(create: () =>	hiltViewModel<VM, Factory> { f -> f.create(repo)
Factory pattern	Factory constructor or manual DI	@AssistedFactory (Hilt generates impl)
Two impls, one interface	Two subclasses or Provider override	@Named + two @Binds OR @AssistedInject